Neurolabs Android SDK Integration Guide

Technical Integration Specification

Document ID: NLB-ANDROID-INTEGRATION-GUIDE
Version: v1.6.10
Date: 2026-08-20
Status: Partner integration reference
Owner: Neurolabs

1. Scope

This guide is for partner Android apps integrating ‘neurolabs-android-sdk‘ directly.

2. Release History

No public API contract has been broken since v1.1.9. New options have been added with backwards-compatible defaults; integrations on v1.1.9 keep working as is. Recommended defaults: strict shelf guidance with ‘validationPreset = IOS_PARITY‘, ‘liveQualityChecksEnabled = true‘, ‘autoCloseAfterCapture = true‘ for single-capture parity flows.

v1.3.2

- Mixpanel ingestion endpoint now resolves via ‘NLAnalyticsDefaults.MIXPANEL_ENDPOINT_EU‘ at SDK construction; events flow into the EU region project as expected. The standalone ‘MixpanelAnalyticsTracker‘ class keeps the US endpoint as its default so partners reusing it for their own US-region projects are unaffected.
- ‘CaptureViewModel.onFinishPressed‘ now claims ‘isFinishing‘ synchronously, wraps every disk read (pending review reload, capture reload, temp-file cleanup) in ‘withContext(Dispatchers.IO)‘, and offloads ‘spillCaptureToDisk‘ from ‘commitCapture‘ - the Done button stays responsive throughout multi-capture sequences and shows an inline ‘CircularProgressIndicator‘ while persistence is in flight.
- Embedded custom-camera demo gained an in-app settings sheet (gear icon, top-right of the camera). Sliders for detection thresholds, capture min / max, live-quality FPS; switches for every boolean knob; SharedPreferences-backed persistence across launches.

v1.3.1

- Fix for an uncatchable native ‘SIGSEGV‘ in ‘com.google.ai.edge.litert.Model.nativeLoadAsset‘ that triggered when the TFLite asset was stored compressed in the APK, missing, or zero length. ‘NLLiteRTYoloDetector‘ now probes the asset with ‘AssetManager.openFd‘ first and, on failure, extracts the asset to ‘noBackupFilesDir‘ and loads it via the file-path overload.

v1.3.0

- Relaxed guidance thresholds (perspective skew floor raised, angle-off warning relaxed) for less aggressive blocking during normal in-store motion. Android lint passes restored.

v1.2.8

- Onboarding flow: same-barcode relookup is allowed after a completed attempt so partners can retake without restarting the session.

v1.2.7

- Lint compatibility fix for product capture; no API change.

v1.2.6

- Onboarding: partner-overridable operations lookup client and capture debug exports (frame dump for offline review).
- Demo: scrollable launcher on small devices, nav-bar inset respected on the product-capture back button.

v1.2.5

- ‘NLNativeCaptureConfig‘ / ‘NLCustomCameraView‘ expose ‘showsSequenceCounterLabel‘ to toggle the on-screen sequence counter.

v1.2.4

- New capture range controls: ‘allowManualFinish‘, ‘minCapturesBeforeDone‘, and ‘maxCaptures‘ plumbed through ‘NLNativeCaptureConfig‘ for bounded multi-capture flows with a Done button.
- iOS-camera parity work: tap-to-focus, lens-switcher hardening, tightened quality validation thresholds.

v1.2.3

- ‘NLNativeCaptureConfig.cameraMode = AR | DEFAULT‘ lets partners pick between the ARKit/ARCore pipeline and the AVFoundation/CameraX 0.5/1 lens-switcher path.
- ‘init‘ accepts ‘analyticsEnabled‘, ‘sentryEnabled‘, ‘sentryDsn‘ so partner apps can opt out of telemetry or point Sentry at their own project.
- Default validation thresholds (blur, glare, perspective) tightened.

v1.2.2

- Internal: Android plugin integration handling refactor; no public API change.

v1.2.1

- iOS camera-style UI parity polish, lens-switcher state machine hardening, demo Sentry auto-init disabled.

v1.2.0

- Internal scaffolding for v1.2.x: capture range controls landed in v1.2.4, lens switcher in v1.2.4-v1.2.5, missions policy preset, retry + idempotency conformance fixture.

v1.1.9 (baseline)

- No public API contract break.
- Default task routing config uses ‘taskUUID‘.
- Recommended custom-camera shelf payload remains strict shelf guidance with ‘validationPreset = IOS_PARITY‘.
- ‘liveQualityChecksEnabled‘ remains the key switch for pill / rotation / warning / error guidance behavior.

- ‘autoCloseAfterCapture = true‘ remains the recommended parity flow for single accepted captures.

3. Requirements

- ‘minSdk 26‘
- JDK 17
- Kotlin 2.x

4. Install

```
// app/build.gradle.kts
dependencies {
    implementation(files("libs/neurolabs-android-sdk-v1.1.x.aar"))
}
```

If you consume a local ‘.aar‘, include required runtime deps (Compose/CameraX/serialization/etc.) or use the provided integration gradle script from the Cordova plugin setup.

5. SDK Initialization + Warmup

```
import ai.neurolabs.sdk.core.*
import kotlinx.coroutines.launch

class MainApp : Application() {
    override fun onCreate() {
        super.onCreate()

        val config = NLConfiguration.production(
            apiKey = BuildConfig.NL_API_KEY,
            taskUUID = BuildConfig.NL_TASK_UUID,
            debugLogging = BuildConfig.DEBUG
        )

        NeurolabsSDKCore.configure(this, config)
    }
}

// e.g. in first Activity/VM
val sdk = NeurolabsSDKCore.shared ?: return
lifecycleScope.launch {
    sdk.initialize(
        modelFileName = "NLBModel.tflite",
        labelsFileName = "labels.json"
    )
    sdk.waitForFullyLoaded()
}
```

6. Queue Management

```
val sdk = NeurolabsSDKCore.shared ?: return

// runtime queue behavior
sdk.setAutoSyncEnabled(true)
sdk.setWifiOnlyUploadsEnabled(false)
sdk.setDeletePhotosOnUploadSuccess(true)
sdk.setMaxQueueSize(200)
sdk.setUploadRetryCount(5)

// lifecycle actions
lifecycleScope.launch {
    sdk.flushQueue()
    sdk.retryFailedUploads()
}

val status = sdk.getQueueStatus()
println("pending=${status.pendingCount} failed=${status.failedCount}")
```

7. Open Custom Camera (Native Capture UI)

```
import ai.neurolabs.sdk.models.*
import ai.neurolabs.sdk.ui.openNativeCaptureUI

val sessionId = UUID.randomUUID().toString()

val config = NLNativeCaptureConfig(
    guidanceMode = NLCaptureGuidanceMode.STRICT,
    validationPreset = NLValidationPreset.IOS_PARITY,
    confidenceThreshold = 0.25f,
    iouThreshold = 0.45f,
    maxCaptures = 5,
    allowManualFinish = true,
    minCapturesBeforeDone = 2,
    showsCapturedCountLabel = false,
    showAlignmentGuidance = true,
    enableValidation = true,
    autoCloseAfterCapture = true,
    showDetections = false,
    showCapturedRegions = false,
    liveQualityChecksEnabled = true,
    liveQualityTargetFps = 6
)

val options = NLNativeCaptureOptions(
    type = NLShelfType.SHELF,
    taskUUID = "<TASK_UUID_FOR_THIS_SESSION>",
    sendDetectionsMetadata = true,
    enablePreviewCropping = true,
    maxImageDimension = 1920,
    imageCompressionQuality = 0.85f
)

sdk.openNativeCaptureUI(
    context = this,
    sessionId = sessionId,
    config = config,
    options = options
)
```

Recommended capture payload notes:

- ‘liveQualityChecksEnabled = true‘ is the key for pill/rotation/warning/error guidance behavior.
- Keep ‘type = NLShelfType.SHELF‘ and ‘guidanceMode = NLCaptureGuidanceMode.STRICT‘ for full shelf guidance checks.
- Keep ‘showDetections = false‘ and ‘showCapturedRegions = false‘ to stay on the custom-camera guidance UI path.
- ‘guidanceMode = NLCaptureGuidanceMode.GUIDANCE‘ should only be used as a temporary fallback if a partner specifically needs looser Android behavior while a strict-guidance regression is being fixed.

8. Multi-Bay Capture (Long Shelves)

Multi-bay capture handles shelves too long for a single photo: the user side-steps along the shelf and captures one overlapping photo per bay, the SDK guides the side-step, dedups the overlap on device, and reports merged product counts across bays in a single session result.

Enable it by setting ‘multiBay‘ on the capture config (‘null‘ = feature off, the default):

```
import ai.neurolabs.sdk.models.*
import ai.neurolabs.sdk.multibay.NLMultiBayOptions
import ai.neurolabs.sdk.ui.openNativeCaptureUI

val config = NLNativeCaptureConfig(
    guidanceMode = NLCaptureGuidanceMode.STRICT,
    multiBay = NLMultiBayOptions(
        minBays = 2,                    // minimum bays before the session can finish (default
 1)
        maxBays = 4,                    // maximum bays per session (default 8, hard cap 8)
        targetOverlapFraction = 0.30,  // fraction of the previous bay kept visible (default
0.30)
        minDetectionConfidence = 0.5,  // confidence floor for merge/count participation (def
ault 0.5)
        onDeviceDedup = true,           // run overlap dedup at session finish (default true)
        generatePreview = true,         // compose the display-only stitched preview (default
true)
        previewMaxDimension = 2048     // preview long-edge cap (default 2048, clamped 512..4
096)
    )
)

sdk.openNativeCaptureUI(
    context = this,
    sessionId = sessionId,
    config = config
)
```

The same ‘multiBay‘ field exists on ‘NLCameraConfiguration‘ for the custom-camera config path, and on the WebView/Cordova bridge payload (camelCase field names identical across iOS, Android, and Cordova; absent fields take the defaults above).

Reading merged results:

- Activity Result / bridge: ‘NLNativeCaptureResultPayload.multiBay‘ (‘NLNativeMultiBayResultSummary?‘, null unless the session ran multi-bay) carries ‘countsByLabel: Map<String, Int>‘, ‘baysCount‘, ‘dedupTrusted‘, and ‘previewImageFileUri‘.
- Compose: ‘NLCaptureScreen‘ / ‘NLCustomCameraView‘ accept an ‘onMultiBayResult: ((NLMultiBayResult) -> Unit)?‘ callback delivering the full result - ‘bays: List<NLBaySummary>‘, ‘mergedProducts: List<NLMergedProduct>‘ (one entry per physical product instance after cross-bay dedup), ‘countsByLabel‘, the stitched preview (‘previewImageData‘), and ‘dedupTrusted‘.
- Use ‘NLMultiBayResult.productCount‘ for a product tally - it sums only ‘sku_single‘ + ‘sku_multipack‘; ‘countsByLabel‘ also carries price/promo/poster buckets, so summing every label over-counts products.

Meaning of ‘dedupTrusted‘:

- ‘true‘ - counts and ‘mergedProducts‘ are on-device deduped: each physical product instance is counted once across overlapping bays.
- ‘false‘ - an adjacent-pair alignment broke, the dedup timed out, or ‘onDeviceDedup‘ was disabled; counts are then the per-bay sum (an upper bound), not a deduped count.

Upload semantics:

- Each bay photo uploads as a normal capture through the operations queue (resumable mission flow, one image per capture).
- On ‘stopSession‘ the mission submit attaches a compact merge summary (‘{bays, counts_by_label, dedup_trusted, ref_homographies, sdk_version}‘) as structured ‘client_metadata.device_dedup‘ plus, transitionally, a ‘notes‘ duplicate for older backends - so the backend can re-dedup authoritatively.
- SDK-managed capture flows record that summary automatically. Only hosts presenting their own custom capture UI must pass ‘NLMultiBayResult.submitNotesJSON()‘ to ‘sdk.setMultiBaySubmitNotes(notes, sessionId)‘ before ‘stopSession‘.

Config-only mode:

- There is no partner-facing in-camera mode toggle. The Single Multi-bay capture-mode chip and sheet are internal demo/debug chrome gated behind ‘NLSDKDebug.internalCameraToolsEnabled‘ (default ‘false‘; never enable it in partner builds). Partners control the mode purely via configuration: ‘multiBay‘ set = multi-bay session, ‘multiBay = null‘ = single-bay session - programmatic ‘multiBay‘ configuration is unaffected by the gate.
- The Neurolabs demo app enables that chrome only in local debug builds; Firebase/release demo builds default to single-bay exactly like partner builds. The multi-bay feature itself is fully available in all builds via configuration - only the demo defaults changed.

9. Post-Processing Hooks

Use Activity Result API and process captures when camera returns.

```
private val captureLauncher = registerForActivityResult(
    NLCaptureActivity.Contract()
) { result ->
    when (result) {
        is NLCaptureActivityResult.Success -> {
            // capture metadata is in result.payload
            val count = result.payload.captures.size
            Log.d("NLB", "camera returned with $count captures")
        }
        is NLCaptureActivityResult.Cancelled -> {
            Log.d("NLB", "camera cancelled")
        }
        is NLCaptureActivityResult.Error -> {
            Log.e("NLB", "camera error: ${result.error.message}")
        }
    }
}
```

10. Delegate/Event Callbacks

```
sdk.delegate = object : NeurolabsSDKDelegate {
    override fun onCaptureResult(result: NLNativeCaptureResult, captureId: String, sessionId:
 String) {}
    override fun onError(error: NLError, sessionId: String?, messageId: String?) {}
    override fun onQueueStatusChange(status: NLQueueStatus) {}
    override fun onLoadingProgress(progress: NLLoadingProgress) {}
    override fun onQueueItemEnqueued(item: NLQueuedItem) {}
    override fun onUploadSucceeded(item: NLQueuedItem) {}
    override fun onUploadFailed(item: NLQueuedItem, error: NLError) {}
}
```

11. On-Device Recognition (v1.7.1)

Per-account, config-gated: your operations API key resolves YOUR org's recognition config and vector dataset - enabling an account is a backend flip, no app change. Every stage degrades gracefully; capture is never blocked by recognition.

```
import ai.neurolabs.sdk.recognition.NLRecognitionBootstrap

val bootstrap = NLRecognitionBootstrap(
    context,
    NLRecognitionBootstrap.Configuration(
        operationsApiKey = "<your operations key>",
        operationsBaseUrl = "https://api.operations.neurolabs.ai/v1",
    ),
)
bootstrap.start(sdk, applicationScope) // one-shot, idempotent
// bootstrap.status (StateFlow)  "provider: ready (N items)" etc.
```

The embedder '.tflite' downloads from your org's snapshot automatically (an embedder-named asset is the fallback). The release AAR carries the Qdrant Edge native libraries; the default index remains the built-in brute-force this release.

Tap-to-product-card: captures expose per-detection matches via 'sdk.recognitionMatches(forCaptureId:)'; pair the SDK's 'DetectionOverlay(onDetectionTap = ...)' with 'NLDetectionInspectorSheet(detection, match, resolver, onDismiss)' and an 'NLProductDetailsResolver' built over 'NLOperationsCatalogClient'. Offline or unresolved products fall back to a UUID + similarity row.

12. Notes

- Per-session upload routing is supported with 'options.taskUUID'.
- 'autoCloseAfterCapture=true' closes after Save from preview/review flow.
- Use 'allowManualFinish = true' with 'minCapturesBeforeDone' and 'maxCaptures' when you want "at least X, up to Y" capture sequences.
- Set 'showsCapturedCountLabel = false' to hide the 'N captured' chip in the top bar.
- For partner apps, avoid passing base64 images across process boundaries.